

OPTION B INSTITUTE FAMILY

M04 - PYTHON FOUNDATIONS

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

16 canonical lessons | full 16-stage teacher contract | protected attempt-first review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE04-01 - Python setup and first program	5
DE04-02 - Variables, data types and operators	7
DE04-03 - Strings and formatting	9
DE04-04 - Conditions and boolean logic	11
DE04-05 - Lists, tuples, sets and dictionaries	13
DE04-06 - Loops and iteration	15
DE04-07 - Functions and scope	17
DE04-08 - Modules, imports and project structure	19
DE04-09 - Exceptions and debugging basics	21
DE04-10 - Files, paths, CSV and JSON I/O	23
DE04-11 - pandas foundations: load, inspect, filter and aggregate	25
DE04-12 - Practical Big-O intuition	27
DE04-13 - Coding drills: lists and strings	29
DE04-14 - Coding drills: dictionaries and sets	31
DE04-15 - Stack, queue, search and sort basics	33
DE04-16 - Integrated Python checkpoint and reproducible script habits	35
Module mastery checklist + revision quick reference	37

Prerequisite / tool map

Write, debug and explain foundational Python programs independently.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE04-01 - Python setup and first program

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Run a Python program from a clean start, distinguish code from output, and use an error message as evidence instead of guessing.

3. Prerequisite / starting state

M03 complete. No WSL or Git required. Install/use Python only because M04 is the first Python module.

4. Picture it in your head

Python is an interpreter: it reads valid Python instructions and evaluates them according to the language rules. A program can be one line or many files.

5. New words before we use them

Python is an interpreter: it reads valid Python instructions and evaluates them according to the language rules. A program can be one line or many files. For a beginner, the safest loop is tiny change -> run -> inspect output/error -> explain what happened.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 0:08:42-0:18:52

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`print("Hello, data engineering")` `print(2 + 3)` # Change one value, run again, and compare the output. Run both lines. Change the message. Create one simple syntax error on purpose, read the exception/line indicator, repair only that defect, then run again from a clean start.

8. Expected result / what you should see

Expected output appears; the error is preserved in your notes; the repaired file runs after closing and reopening the interpreter/editor.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If nothing runs, first prove which Python executable you are using (``py --version`` or ``python --version``). If syntax fails, read the last error line and inspect quotes, parentheses and indentation near the reported location.

11. If you get stuck

If nothing runs, first prove which Python executable you are using (``py --version`` or ``python --version``). If syntax fails, read the last error line and inspect quotes, parentheses and indentation near the reported location.

12. Small independent task

- With the video closed, create ``p01_first_program.py`` that prints your name, ``Standalone DE 2026``, and the result of `7 *`
- Create and repair one syntax error, preserving the first failed output.

13. Validate

Expected output appears; the error is preserved in your notes; the repaired file runs after closing and reopening the interpreter/editor. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

What does Python execute, what is output, and how did the error message tell you where to inspect?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Create a fresh file that prints a different label and calculates $9 * 6$. Introduce a different syntax error from your first attempt, explain it, then repair it without opening Review R01.

16. Evidence / mastery + exact next route

p01_first_program.py + first failed output + repaired output + 3-5 sentence explain-back. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_PYTHON_PRACTICE_CANONICAL.py Exact next route: DE04-02

DE04-02 - Variables, data types and operators

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Represent identifiers, counts, money-like values, status text, booleans and missing values with intentional Python types and valid operators.

3. Prerequisite / starting state

DE04-01 mastered or explicitly remediated.

4. Picture it in your head

A variable is a name bound to a value. The value has a type, and the type affects which operations are meaningful.

5. New words before we use them

A variable is a name bound to a value. The value has a type, and the type affects which operations are meaningful. `4` is an integer; `850.0` is a floating-point number; `"O2001"` is text even though it contains digits; `True`/`False` are booleans; and `None` represents the absence of a Python value.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 0:18:52-0:48:41

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
order_id = "O2001" qty = 4 unit_price = 850.0 gross = qty * unit_price is_large = gross >= 3000
print(order_id, gross, is_large, type(gross))
```

Create examples of int, float, str, bool and None. Print each value and its type. Then calculate `qty * unit_price` and one boolean comparison.

8. Expected result / what you should see

Types match business meaning; arithmetic is applied only to measures; comparison output is boolean; you can explain why an identifier remains text.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Print the value and `type(value)` before changing code. If concatenation/arithmetic fails, inspect whether text needs parsing or whether the field should remain text.

11. If you get stuck

Print the value and `type(value)` before changing code. If concatenation/arithmetic fails, inspect whether text needs parsing or whether the field should remain text.

12. Small independent task

Create `order_id="O3107"`, `qty=4`, `unit_price=850.0`, `status="Completed"`. Calculate gross sales and a boolean for `gross >= 3000`. Explain the type of every variable.

13. Validate

Types match business meaning; arithmetic is applied only to measures; comparison output is boolean; you can explain why an identifier remains text. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Why are identifiers often strings, and what is the difference between ``=`` and ``==``?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Use `shipment_id="S009"`, `cartons=7`, `unit_cost=425.5`, `delivered=False`. Calculate cost and ``needs_review = (cost > 2500 and not delivered)``.

16. Evidence / mastery + exact next route

Saved code + printed values/types + explanation of identifier vs measure. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: `M04_PYTHON_PRACTICE_CANONICAL.py` Exact next route: DE04-03

DE04-03 - Strings and formatting

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Clean whitespace/case deliberately, build readable labels with f-strings, and preserve the original when normalization might alter business meaning.

3. Prerequisite / starting state

DE04-02.

4. Picture it in your head

Strings are sequences of characters. Common cleaning methods such as ``.strip()``, ``.lower()``, ``.upper()`` and ``.title()`` return new strings; they do not magically define the correct business value.

5. New words before we use them

Strings are sequences of characters. Common cleaning methods such as ``.strip()``, ``.lower()``, ``.upper()`` and ``.title()`` return new strings; they do not magically define the correct business value. Cleaning should therefore be deliberate and auditable.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 0:48:41–1:05:06

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`raw = " NORTH zone "` `clean = raw.strip().title()` `print(repr(raw))` `print(f"Region: {clean}")` Apply `strip/lower/upper/title` to sample labels and compare `repr`` before/after. Build two f-string labels.

8. Expected result / what you should see

Whitespace changes are visible; casing is intentional; original can still be compared; f-string output matches the required label exactly.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If output looks identical, print `repr(value)``. If title-case changes a name incorrectly, stop using it as a blanket business rule and define an explicit mapping instead.

11. If you get stuck

If output looks identical, print `repr(value)``. If title-case changes a name incorrectly, stop using it as a blanket business rule and define an explicit mapping instead.

12. Small independent task

Normalize " EAST hub " to "East Hub" and create ``Hub: East Hub | Status: Ready``. Preserve and print the raw value too.

13. Validate

Whitespace changes are visible; casing is intentional; original can still be compared; f-string output matches the required label exactly. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Why is string cleaning not the same as business standardization?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Normalize " west DEPOT " to an agreed label and create a separate message containing the original and normalized values.

16. Evidence / mastery + exact next route

Raw + normalized output + code + one caveat about automatic casing. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_PYTHON_PRACTICE_CANONICAL.py Exact next route: DE04-04

DE04-04 - Conditions and boolean logic

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Translate a business rule into if/elif/else with correct AND/OR behavior and boundary tests.

3. Prerequisite / starting state

DE04-03.

4. Picture it in your head

A condition is an expression whose result is truthy/falsey; explicit comparisons usually make business rules easier to audit. `and` requires both sides to be true, while `or` requires at least one.

5. New words before we use them

A condition is an expression whose result is truthy/falsey; explicit comparisons usually make business rules easier to audit. `and` requires both sides to be true, while `or` requires at least one. Parentheses are useful when a rule combines several conditions.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 1:26:20–1:34:24

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

`status = "Completed" amount = 2500 if status == "Completed" and amount >= 2500: label = "priority" else: label = "standard" print(label)` Test Completed/2800, Completed/2499, Completed/2500 and Pending/3000. State the expected label before running.

8. Expected result / what you should see

Boundary cases behave exactly as the written rule; a high Pending amount does not become priority.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Write the English rule in one sentence, circle the words AND/OR, then translate one clause at a time. Print intermediate booleans if needed.

11. If you get stuck

Write the English rule in one sentence, circle the words AND/OR, then translate one clause at a time. Print intermediate booleans if needed.

12. Small independent task

Classify an order as `priority` only if status is Completed and amount $\geq$ 2500; otherwise `standard`. Use at least four boundary cases.

13. Validate

Boundary cases behave exactly as the written rule; a high Pending amount does not become priority. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

When would replacing AND with OR change the eligible rows?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Classify a shipment as `escalate` only if `delayed_days >= 2` AND `priority_customer` is True. Test 1/2/3 days and both customer flags.

16. Evidence / mastery + exact next route

Rule sentence + code + boundary-case table/output + explain-back. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_PYTHON_PRACTICE_CANONICAL.py Exact next route: DE04-05

DE04-05 - Lists, tuples, sets and dictionaries

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose list, tuple, set or dict based on order, mutability, uniqueness/membership and key-value lookup needs.

3. Prerequisite / starting state

DE04-04.

4. Picture it in your head

A list is an ordered mutable sequence. A tuple is an ordered sequence commonly used for a fixed-size record or values that should not be casually changed.

5. New words before we use them

A list is an ordered mutable sequence. A tuple is an ordered sequence commonly used for a fixed-size record or values that should not be casually changed. A set stores unique hashable values and is excellent for membership/duplicate detection.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 1:34:24–2:12:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

pipeline = ["extract", "transform", "load"] coords = (23.3, 85.3) ids = {"O1", "O2", "O1"} manager = {"East": "Asha", "West": "Rohit"} print(pipeline, coords, ids, manager["East"]) Build one list, tuple, set and dict from a small operations example. For each, say what property made it appropriate.

8. Expected result / what you should see

Choices match the requirement; duplicates in the set collapse intentionally; dictionary keys are meaningful/unique.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Ask four questions: do I need order? mutation? unique membership? key-to-value lookup? Choose the simplest structure that answers the requirement.

11. If you get stuck

Ask four questions: do I need order? mutation? unique membership? key-to-value lookup? Choose the simplest structure that answers the requirement.

12. Small independent task

Map three regions to managers with a dict; deduplicate order IDs with a set; keep a three-step pipeline in a list; store one fixed coordinate pair in a tuple.

13. Validate

Choices match the requirement; duplicates in the set collapse intentionally; dictionary keys are meaningful/unique. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Give one realistic case where a set is better than a list and one where a dict is better than both.

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Represent warehouse zones and supervisors, a unique set of failed batch IDs, a processing sequence, and a fixed `(year, month)` period.

16. Evidence / mastery + exact next route

Four structures + printed behavior + structure-choice explanation. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_PYTHON_PRACTICE_CANONICAL.py Exact next route: DE04-06

DE04-06 - Loops and iteration

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Iterate through collections, maintain explicit accumulators/counters and validate a loop with an independent control.

3. Prerequisite / starting state

DE04-05.

4. Picture it in your head

A `for` loop visits items from an iterable one at a time. State that changes across iterations - such as a running total, count or output list - should be initialized before the loop and updated at the correct indentation level.

5. New words before we use them

A `for` loop visits items from an iterable one at a time. State that changes across iterations - such as a running total, count or output list - should be initialized before the loop and updated at the correct indentation level. While learning, a clear loop is often better than compressing everything into one line.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 2:21:06-2:36:31

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
values = [1700, 1350, 1100] total = 0
for value in values:
    total += value
print(total)
Total a list manually, count values >= 1500 and collect values below 1000. Only after the loop, compare the total with `sum()`.
```

8. Expected result / what you should see

Each input is processed once; accumulator starts at the correct value; control total matches; output list contains only eligible values.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Print the accumulator inside the loop to see each step. Check indentation and initialization before changing the algorithm.

11. If you get stuck

Print the accumulator inside the loop to see each step. Check indentation and initialization before changing the algorithm.

12. Small independent task

From [1700,1350,1100,850,1800], compute total, count >=1500, and collect values below 1000 using a loop.

13. Validate

Each input is processed once; accumulator starts at the correct value; control total matches; output list contains only eligible values. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

What state changes on each iteration, and how did you independently validate the final state?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: From [925, 2100, 1750, 600, 2400, 1100], calculate total, count ≥ 1800 and collect values under 1000.

16. Evidence / mastery + exact next route

Loop code + step/output evidence + independent total check. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_PYTHON_PRACTICE_CANONICAL.py Exact next route: DE04-07

DE04-07 - Functions and scope

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Write small reusable functions with explicit parameters, returned values, predictable behavior and no accidental global dependencies.

3. Prerequisite / starting state

DE04-06.

4. Picture it in your head

A function gives a name to reusable logic. Parameters are inputs supplied by the caller; local variables exist inside the function; ``return`` sends a value back to the caller.

5. New words before we use them

A function gives a name to reusable logic. Parameters are inputs supplied by the caller; local variables exist inside the function; ``return`` sends a value back to the caller. ``print`` only displays something and is not a substitute for returning data needed by later code.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 2:52:12–3:02:03

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
def gross_sales(qty, unit_price): return qty * unit_price
print(gross_sales(2, 850))
print(gross_sales(3, 450))
```

Write two small functions, call each with at least two input sets, and show the difference between a function that prints and one that returns.

8. Expected result / what you should see

Different inputs produce expected results; returned value can be assigned/reused; no global variable is needed for the calculation.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Check three things separately: parameters, calculation, and return path. Temporarily print parameters at the function boundary if the wrong value is entering.

11. If you get stuck

Check three things separately: parameters, calculation, and return path. Temporarily print parameters at the function boundary if the wrong value is entering.

12. Small independent task

Write ``summarize_order(order_id, qty, unit_price)`` returning a dict with ``order_id`` and ``gross_sales``. Test two inputs.

13. Validate

Different inputs produce expected results; returned value can be assigned/reused; no global variable is needed for the calculation. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Why is ``return`` important for data pipelines, and what risk does hidden global state create?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Write ``shipment_cost(shipment_id, cartons, cost_per_carton)`` returning a dict. Test three inputs including zero cartons.

16. Evidence / mastery + exact next route

Function code + at least two calls + reusable return proof + explain-back. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_PYTHON_PRACTICE_CANONICAL.py Exact next route: DE04-08

DE04-08 - Modules, imports and project structure

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Split reusable helpers from an entry script, import them from the intended project root, and explain why folder/run location affects imports.

3. Prerequisite / starting state

DE04-07. Local folder-based Python becomes required here if you used only a browser notebook earlier.

4. Picture it in your head

A Python module is a `.py` file. Importing lets one file use names defined in another file instead of duplicating code.

5. New words before we use them

A Python module is a `.py` file. Importing lets one file use names defined in another file instead of duplicating code. A small project can have an entry script plus helper module; later modules will formalize larger package structure.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 3:12:46–3:28:16

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
# stage04_helpers.py def gross_sales(qty, unit_price): return qty * unit_price #
M04_MODULE_DEMO_CANONICAL.py from stage04_helpers import gross_sales print(gross_sales(2,
850)) Complete the two TODO helpers in `stage04_helpers.py`, then run
`M04_MODULE_DEMO_CANONICAL.py` from the practice-files folder.
```

8. Expected result / what you should see

Import succeeds from the documented folder; helper logic lives only in the helper module; running the entry file prints expected results.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Confirm filenames and current working directory first. Run from the folder containing both files. Do not rename files halfway through without updating imports.

11. If you get stuck

Confirm filenames and current working directory first. Run from the folder containing both files. Do not rename files halfway through without updating imports.

12. Small independent task

Create `helpers.py` with one reusable calculation and a separate `run_demo.py` that imports it. Prove a clean run after closing/reopening the terminal/editor.

13. Validate

Import succeeds from the documented folder; helper logic lives only in the helper module; running the entry file prints expected results. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

What is a module, what does import do, and why can the same import fail from the wrong project/run location?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Create a fresh `shipment_helpers.py` plus `run_shipment_demo.py` with different function names/inputs; do not copy your first file names.

16. Evidence / mastery + exact next route

Two-file project + clean-run command/output + brief module/import explanation. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: `stage04_helpers.py`, `M04_MODULE_DEMO_CANONICAL.py` Exact next route: DE04-09

DE04-09 - Exceptions and debugging basics

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Read a traceback, isolate one defect at a time, and handle only expected/recoverable exceptions without hiding unrelated failures.

3. Prerequisite / starting state

DE04-08.

4. Picture it in your head

A traceback shows the path of calls leading to an exception. Start at the final exception type/message, then find the first relevant line in your code.

5. New words before we use them

A traceback shows the path of calls leading to an exception. Start at the final exception type/message, then find the first relevant line in your code. Fix one root cause and rerun so you know which change affected the behavior.

6. Useful video / source

Source: Corey Schafer — Python Try/Except Error Handling

Assigned scope: Full focused lesson

How to use it: Watch the focused exceptions lesson, type the examples, then debug the course task without the video.

Note: Older source, but Python try/except fundamentals are stable; final audit will still recheck availability.

[Open source](#)

7. Follow with me once

`raw = "12.5" try: qty = int(raw) except ValueError: print(f"Invalid integer quantity: {raw!r}")` Run ``M04_DEBUG_ME_CANONICAL.py`` unchanged. Save the first traceback. Repair one problem, rerun, and repeat until valid input works and invalid/missing values fail visibly.

8. Expected result / what you should see

First error is preserved; each fix is isolated; expected bad input has a useful message; no bare/broad swallowing of exceptions.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you are changing many lines at once, restore the file and work one traceback at a time. If unsure what to catch, first let the error occur and identify its specific exception type.

11. If you get stuck

If you are changing many lines at once, restore the file and work one traceback at a time. If unsure what to catch, first let the error occur and identify its specific exception type.

12. Small independent task

Write a ``parse_quantity(raw)`` function that returns an int for valid text and raises or reports a clear problem for invalid text without hiding unrelated exceptions.

13. Validate

First error is preserved; each fix is isolated; expected bad input has a useful message; no bare/broad swallowing of exceptions. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Why is swallowing every exception dangerous in a data pipeline?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Write ``parse_unit_cost(raw)`` accepting numeric text such as "425.50" and rejecting "unknown" with a specific, visible error path.

16. Evidence / mastery + exact next route

Original traceback + repaired code/output + one deliberate bad-input proof + explanation. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_DEBUG_ME_CANONICAL.py Exact next route: DE04-10

DE04-10 - Files, paths, CSV and JSON I/O

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build file paths predictably, read/write text and structured JSON, and inspect CSV/JSON without confusing file format with business grain.

3. Prerequisite / starting state

DE04-09.

4. Picture it in your head

`pathlib.Path`` represents filesystem paths without manual slash juggling. Build course paths from the project/script location when you need repeatability.

5. New words before we use them

`pathlib.Path`` represents filesystem paths without manual slash juggling. Build course paths from the project/script location when you need repeatability. Text files should use an explicit encoding such as UTF-8.

6. Useful video / source

Source: Corey Schafer — Python File Objects

Assigned scope: Full focused file I/O lesson; Teaching Book adds CSV/JSON specifics

How to use it: Use for file/path I/O mechanics. The course remains authoritative for CSV/JSON engineering patterns and validation.

Note: Older but stable Python file-I/O mechanics.

[Open source](#)

7. Follow with me once

```
from pathlib import Path
import json
root = Path(__file__).parent
out = root / "run_manifest.json"
out.write_text(json.dumps({"run_id": "R1", "rows": 3}, indent=2), encoding="utf-8")
loaded = json.loads(out.read_text(encoding="utf-8"))
assert loaded["rows"] == 3
# Use pathlib to locate the packaged retail CSV. Write a small JSON run manifest, read it back, and compare each expected key/value.
```

8. Expected result / what you should see

Path does not depend on a random working folder; UTF-8 is explicit; input raw file remains unchanged; JSON read-back matches what was written.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Print the resolved path before assuming the file is missing. If JSON parsing fails, inspect the exact file text and error location rather than rewriting it blindly.

11. If you get stuck

Print the resolved path before assuming the file is missing. If JSON parsing fails, inspect the exact file text and error location rather than rewriting it blindly.

12. Small independent task

Write JSON containing source filename, expected row count and status; reload and validate all three values. Also read the CSV header with the standard `csv`` module.

13. Validate

Path does not depend on a random working folder; UTF-8 is explicit; input raw file remains unchanged; JSON read-back matches what was written. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

What problem does pathlib solve, and why should you read back a generated file before trusting it?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Create a fresh manifest for `M04\_RETEST\_SHIPMENTS\_CANONICAL.csv`, then read its CSV header and validate the required fields without pandas.

16. Evidence / mastery + exact next route

Script + generated JSON + read-back assertions/output + resolved input path. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_JSON_IO_SAMPLE.json, M04_RETAIL_ORDERS_CANONICAL.csv, M04_RETEST_SHIPMENTS_CANONICAL.csv Exact next route: DE04-11

DE04-11 - pandas foundations: load, inspect, filter and aggregate

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Load a CSV into pandas, inspect shape/types, filter rows deliberately, calculate a column, aggregate at a stated grain and reconcile totals.

3. Prerequisite / starting state

DE04-10. Install pandas only now, at its first required use.

4. Picture it in your head

A pandas DataFrame is a table-like in-memory object. Start by inspecting shape, columns, sample rows and dtypes; do not immediately transform unfamiliar data.

5. New words before we use them

A pandas DataFrame is a table-like in-memory object. Start by inspecting shape, columns, sample rows and dtypes; do not immediately transform unfamiliar data. A boolean filter returns only rows meeting a condition.

6. Useful video / source

Source: Luke Barousse — Python for Data Analytics

Assigned scope: 4:22:56–5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

```
import pandas as pd
df = pd.read_csv("M04_RETAIL_ORDERS_CANONICAL.csv")
df["sales"] = df["qty"] * df["unit_price"]
completed = df.loc[df["status"].eq("Completed")].copy()
by_region = completed.groupby("region", as_index=False)["sales"].sum()
assert by_region["sales"].sum() == completed["sales"].sum()
Inspect shape/columns/dtypes/head. Calculate sales. Filter Completed.
Group by region. Save row counts and reconcile grouped sum to a separate Completed total.
```

8. Expected result / what you should see

Canonical controls: input rows=25; Completed rows=23; Completed sales=39320.00; missing sales_rep=1. Regional Completed totals: East 11700, North 9310, South 10210, West 8100.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If totals differ, compare in stages: input row count -> filter count -> calculated sales -> grouped total. Do not add `dropna` or `fillna` merely to make numbers match.

11. If you get stuck

If totals differ, compare in stages: input row count -> filter count -> calculated sales -> grouped total. Do not add `dropna` or `fillna` merely to make numbers match.

12. Small independent task

Return Completed sales by region, top 3 Completed orders by sales, and rows with missing sales_rep. Write the result grain for each output.

13. Validate

Canonical controls: input rows=25; Completed rows=23; Completed sales=39320.00; missing sales_rep=1. Regional Completed totals: East 11700, North 9310, South 10210, West 8100. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Why can a DataFrame look correct while still answering the wrong business question?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Using the shipments retest CSV, calculate shipment_value, keep Delivered only, aggregate by hub, list top 3 Delivered shipments and count missing coordinator values. Use the separate retest controls only after your query is complete.

16. Evidence / mastery + exact next route

Code + shape/dtype note + filter count + group result + independent reconciliation + output-grain statements. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_RETAIL_ORDERS_CANONICAL.csv, M04_RETEST_SHIPMENTS_CANONICAL.csv Exact next route: DE04-12

DE04-12 - Practical Big-O intuition

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Recognize practical $O(1)$, $O(n)$ and $O(n^2)$ growth patterns and choose obviously better membership/counting structures without claiming exact runtime.

3. Prerequisite / starting state

DE04-11.

4. Picture it in your head

Big-O describes how resource use grows as input size grows; it is not a stopwatch prediction. A single scan over n records is $O(n)$.

5. New words before we use them

Big-O describes how resource use grows as input size grows; it is not a stopwatch prediction. A single scan over n records is $O(n)$. A nested full scan can become $O(n^2)$.

6. Useful video / source

Source: freeCodeCamp/Jovian — Data Structures and Algorithms in Python

Assigned scope: 00:50:52 — Big-O complexity

How to use it: Watch only the Big-O block needed for practical $O(1)/O(n)/O(n^2)$ intuition; do not turn this into a competitive-programming detour.

Note: Long course used as a precise chapter source.

[Open source](#)

7. Follow with me once

```
seen = set()
for order_id in ["O1", "O2", "O2"]:
    if order_id in seen:
        print("duplicate", order_id)
    seen.add(order_id)
```

Compare repeated membership checks against a list and a set. Describe expected growth rather than timing one tiny example and declaring a universal winner.

8. Expected result / what you should see

Explanation correctly distinguishes list scan from average set lookup and labels a simple nested full scan as $O(n^2)$.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Count how many times the inner operation may run as n grows. Ignore small constant factors at this conceptual level.

11. If you get stuck

Count how many times the inner operation may run as n grows. Ignore small constant factors at this conceptual level.

12. Small independent task

Explain why a set helps repeated duplicate-ID checks; write one nested-loop $O(n^2)$ example you would avoid for large data.

13. Validate

Explanation correctly distinguishes list scan from average set lookup and labels a simple nested full scan as $O(n^2)$. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

What does Big-O tell you, and what does it not tell you?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Compare "for each shipment, scan every known failed ID" with building a failed-ID set once. Explain the growth of both approaches.

16. Evidence / mastery + exact next route

Short code/example + growth explanation + one DE-oriented choice. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_CODING_DRILLS_CANONICAL.py Exact next route: DE04-13

DE04-13 - Coding drills: lists and strings

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Solve small list/string problems from a blank function body, state edge cases and prove results with tests.

3. Prerequisite / starting state

DE04-12. Keep answer keys/AI solving closed during the genuine attempt.

4. Picture it in your head

Coding fluency grows by translating a requirement into input/output examples, a simple plan, code and tests. For these drills, the absence of a tutorial video is intentional.

5. New words before we use them

Coding fluency grows by translating a requirement into input/output examples, a simple plan, code and tests. For these drills, the absence of a tutorial video is intentional. First write one or two examples and the plain-English steps you expect the function to perform.

6. Useful video / source

Source: Independent practice / protected assessment

How to use it: Keep videos, answers and AI solving closed during the attempt. Use review only after saving a genuine attempt.

Note: No tutorial during this task by design. Attempt it independently, then use the locked review/remediation flow.

Disposition: COURSE_BUILT / BOOK_ONLY for this lesson; no filler link is required.

7. Follow with me once

Warm-up: write a plain-English algorithm for reversing word order while deciding how to treat extra spaces. Then implement only after the plan is saved. Warm-up: write a plain-English algorithm for reversing word order while deciding how to treat extra spaces. Then implement only after the plan is saved.

8. Expected result / what you should see

Run ``M04_CODING_DRILLS_VISIBLE_TESTS.py``; supplied tests pass and you add at least one edge case of your own.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Shrink the failing input. Print intermediate values. Return to the input/output contract before changing several lines at once.

11. If you get stuck

Shrink the failing input. Print intermediate values. Return to the input/output contract before changing several lines at once.

12. Small independent task

Complete ``reverse_words`` and ``count_vowels`` in ``M04_CODING_DRILLS_CANONICAL.py``, then add one edge-case test for each.

13. Validate

Run ``M04_CODING_DRILLS_VISIBLE_TESTS.py``; supplied tests pass and you add at least one edge case of your own. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Describe one drill from input -> algorithm -> output -> edge case without reading your code.

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: In ``M04_FRESH_RETEST_TODO.py``, implement a changed ``normalize_words(text)`` task that strips extra spaces, lowercases tokens and returns them as a list. Add tests for blank and multi-space text.

16. Evidence / mastery + exact next route

Saved first attempt + test output + invented edge cases + explain-back. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: `M04_CODING_DRILLS_CANONICAL.py`, `M04_CODING_DRILLS_VISIBLE_TESTS.py` Exact next route: DE04-14

DE04-14 - Coding drills: dictionaries and sets

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use dictionaries for counting/aggregation and sets for uniqueness/membership while preserving deterministic requirements.

3. Prerequisite / starting state

DE04-13.

4. Picture it in your head

A dictionary is a natural accumulator when each key needs a count or total. `counts[key] = counts.get(key, 0) + 1`` is a common pattern.

5. New words before we use them

A dictionary is a natural accumulator when each key needs a count or total. `counts[key] = counts.get(key, 0) + 1`` is a common pattern. A set is natural when the question is only whether a value has been seen or which unique values exist.

6. Useful video / source

Source: Independent practice / protected assessment

How to use it: Keep videos, answers and AI solving closed during the attempt. Use review only after saving a genuine attempt.

Note: No tutorial during this task by design. Attempt it independently, then use the locked review/remediation flow.

Disposition: COURSE_BUILT / BOOK_ONLY for this lesson; no filler link is required.

7. Follow with me once

`regions = ["East", "West", "East"] counts = {} for region in regions: counts[region] = counts.get(region, 0) + 1 print(counts)` Count three region labels with a dict and detect a repeated ID with a set. Explain why a nested loop is unnecessary.

8. Expected result / what you should see

Visible tests pass; output contract is deterministic; repeated duplicates are handled exactly as stated.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Write the expected output for a tiny list by hand. If you need a count/total per key, use a dict accumulator; if only membership matters, start with a set.

11. If you get stuck

Write the expected output for a tiny list by hand. If you need a count/total per key, use a dict accumulator; if only membership matters, start with a set.

12. Small independent task

Complete ``word_counts``, ``first_duplicate`` and ``sales_by_region`` in the drill file, then run visible tests and add one edge case each.

13. Validate

Visible tests pass; output contract is deterministic; repeated duplicates are handled exactly as stated. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

When does a dict hold information that a set cannot?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Implement ``count_statuses`` and ``unique_repeated_batch_ids`` on changed warehouse inputs inside the fresh-retest workspace.

16. Evidence / mastery + exact next route

First attempt + passing visible tests + edge-case tests + data-structure rationale. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: `M04_CODING_DRILLS_CANONICAL.py`, `M04_CODING_DRILLS_VISIBLE_TESTS.py`
Exact next route: DE04-15

DE04-15 - Stack, queue, search and sort basics

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Recognize LIFO/FIFO behavior, implement linear search, explain binary-search sorted-data precondition and sort without accidentally mutating an input when preservation is required.

3. Prerequisite / starting state

DE04-14.

4. Picture it in your head

A stack removes the most recently added item first (LIFO). A queue removes the earliest added item first (FIFO); `collections.deque` supports efficient appends/pops from both ends.

5. New words before we use them

A stack removes the most recently added item first (LIFO). A queue removes the earliest added item first (FIFO); `collections.deque` supports efficient appends/pops from both ends. Linear search checks items until it finds a target or reaches the end.

6. Useful video / source

Source: freeCodeCamp/Jovian — Data Structures and Algorithms in Python

Assigned scope: 1:24:57 — binary vs linear search + 5:16:47 — sorting; course book adds light stack/queue scope

How to use it: Use only the mapped search/sort portions. Stack/queue remain a lightweight course concept; no giant DSA diversion.

Note: Support, not full coverage of every subtopic in the lesson.

[Open source](#)

7. Follow with me once

```
from collections import deque
stack = []
stack.append("extract"); stack.append("transform")
print(stack.pop())
q = deque(["job1", "job2"]); print(q.popleft())
values = [7,2,9,1,5]; ordered = sorted(values)
print(values, ordered)
# Use a list as stack, deque as queue, run a linear-search function
# and compare `sorted` vs `.sort()` behavior.
```

8. Expected result / what you should see

LIFO/FIFO outputs are correct; original list is preserved when required; linear search returns index/-1; binary search precondition is stated.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Act out the structure with 3 labels. For search, write what happens to each candidate position. For sorting, print both original and result.

11. If you get stuck

Act out the structure with 3 labels. For search, write what happens to each candidate position. For sorting, print both original and result.

12. Small independent task

Push extract/transform/load and pop once; create a FIFO queue example; sort `[7,2,9,1,5]` without mutating original; implement `linear_search`.

13. Validate

LIFO/FIFO outputs are correct; original list is preserved when required; linear search returns index/-1; binary search precondition is stated. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Why is binary search invalid on arbitrary unsorted input, and when does ``sorted()`` protect you from accidental mutation?

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: Use tasks ["ingest","validate","publish"] for stack/queue contrasts and search target 12 inside [4,12,7,3]. Sort a copy only.

16. Evidence / mastery + exact next route

Stack/queue/search/sort code + output + 60-second explanation. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_CODING_DRILLS_CANONICAL.py Exact next route: DE04-16

DE04-16 - Integrated Python checkpoint and reproducible script habits

1. Why it matters

This matters because later Python Foundations work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Combine core Python skills into a repeatable folder-based program with explicit input validation, deterministic output, visible failure behavior and a documented run command.

3. Prerequisite / starting state

DE04-01..15 evidence complete. Keep the PyPA reference closed until the first checkpoint attempt is saved.

4. Picture it in your head

A reproducible script can be run again from known inputs and produce explainable outputs without hidden notebook state. Keep inputs explicit, split logic into small functions, validate required fields before calculating, fail visibly when the input contract is broken, and write deterministic output.

5. New words before we use them

A reproducible script can be run again from known inputs and produce explainable outputs without hidden notebook state. Keep inputs explicit, split logic into small functions, validate required fields before calculating, fail visibly when the input contract is broken, and write deterministic output. Project structure communicates how to rerun the work.

6. Useful video / source

Source: Python Packaging User Guide — Packaging Python Projects

Assigned scope: Current project layout: README, pyproject.toml, src/, tests/; use as a structure reference after your first checkpoint attempt

How to use it: Attempt the integrated checkpoint first. Use this current PyPA guide only to repair reproducibility/project-structure weaknesses; it is not a solution walkthrough.

Note: Keeps the checkpoint independent while giving a current official reference for reproducible Python project structure.

[Open source](#)

7. Follow with me once

No answer walkthrough. Read the TODO/rubric, state the input/output contract, then implement one function at a time. No answer walkthrough. Read the TODO/rubric, state the input/output contract, then implement one function at a time.

8. Expected result / what you should see

Clean run succeeds; controls reconcile; bad/missing input fails visibly; raw input is unchanged; output is deterministic; documented run command works from a fresh terminal.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Split failures by boundary: path -> parse -> required columns -> calculation/filter -> aggregation -> output write -> rerun. Repair one boundary at a time.

11. If you get stuck

Split failures by boundary: path -> parse -> required columns -> calculation/filter -> aggregation -> output write -> rerun. Repair one boundary at a time.

12. Small independent task

Complete the checkpoint: load/validate retail CSV, calculate sales, keep Completed, produce regional totals + top 3 + missing-rep count, write deterministic JSON, prove one bad-input failure, and document the run command.

13. Validate

Clean run succeeds; controls reconcile; bad/missing input fails visibly; raw input is unchanged; output is deterministic; documented run command works from a fresh terminal. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

Explain the program from input contract to validation to output, including one failure mode and how a second person can reproduce it.

15. Protected review + changed retry

Save the genuine independent attempt first. If weak, open only the matching protected Review section. Repair, close it, then do the changed retest without copying. Changed retest if assigned: If assigned, complete `M04\_FRESH\_CHECKPOINT\_TODO.py` on the changed shipments fixture with R16 closed; preserve the original attempt.

16. Evidence / mastery + exact next route

First attempt + controls + failure proof + JSON output + run transcript + explain-back. Mastery: changed task passes validation + explain-back. Work corrected with Review open is repair evidence, not mastery. Practice files: M04_CHECKPOINT_TODO.py, M04_RETAIL_ORDERS_CANONICAL.csv, M04_FRESH_CHECKPOINT_TODO.py, M04_RETEST_SHIPMENTS_CANONICAL.csv Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.